



Department of Electrical and Computer Engineering
Object Oriented Programming

Title of Project
Airport Management System

Submitted By

M.ShahzaibAli 2500762

ManzarMehmood 2500770

Bilal Amer
2500784

Submitted To
Engr AyeshaTahir

Class:BCE-2A

Date: 31 May,2025

1. Introduction:

Brief Description:

The Airport Management System is a C++ based management project designed to handle flight ticket booking, cancellation, ticket modification, passenger records and booking statistics. The system provides an efficient way to manage airport ticket operations through a menu-driven interface.

Why we Selected this Topic: AirPort Management is a real world problem .Using C++ (OOP) to make this management project requires critical thinking and logical understanding in short it strengthens the skills and understanding.

Objectives:

- Efficiently manage flight reservations and passenger records.
- Reduce manual workload and record management errors.
- Provide quick access to booking information.
- Allow ticket cancellation and booking modification.
- Store records permanently using file handling.

2. Tools and Technologies used

Tools/Technology	Description
Programming Language	C++
IDE	DEV C++
Storage	Text File Handling
Concepts	OOP, Inheritance, Polymorphism, Composition, Friend Functions

3. Outcomes of this Management System:

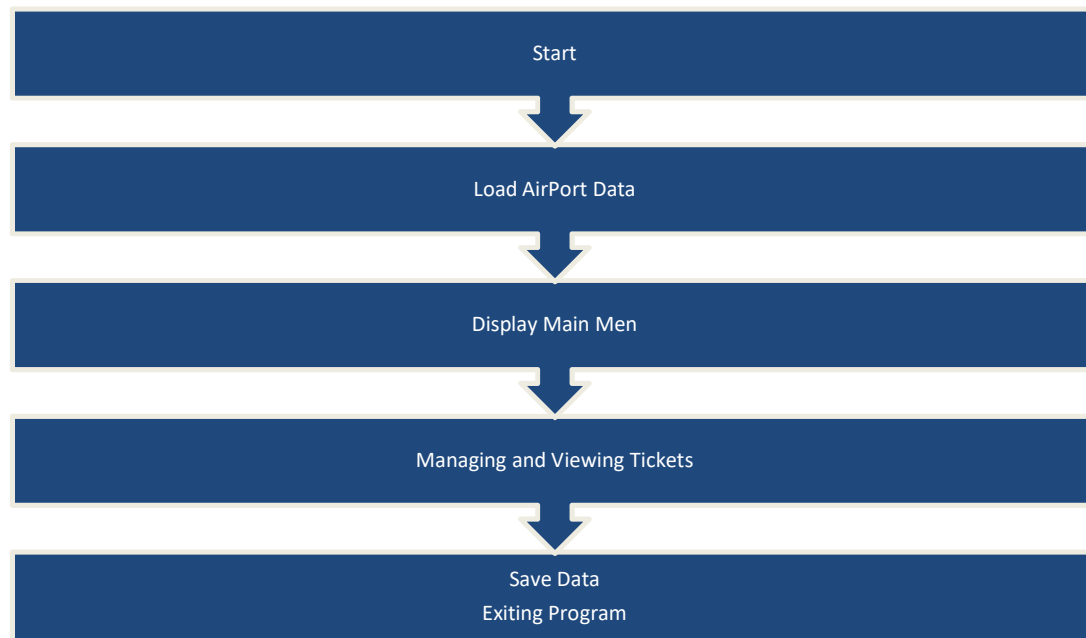
- **Flight Ticket Booking:** Users/Passengers can book flight tickets by entering basic information such as departure city, arrival city, flight date, flight time, passenger name, and CNIC details. After confirmation, a flight ticket receipt is generated and displayed.
- **Flight Ticket Cancellation:** Users/Passengers can cancel their booked flight ticket by entering the Ticket ID issued at the time of booking. The ticket record is removed successfully from the system.
- **Viewing Booked Tickets:** Users/Passengers can view all booked flight tickets with complete details including passenger information, route, flight schedule, class, and fare.
- **Flight Booking Modification:** Users/Passengers can update their existing bookings by changing flight date, flight time, or destination city. The updated booking information is saved in the system.
- **Booking Statistics Report:** The system provides a booking statistics report showing the total number of booked tickets and the number of passengers scheduled for each flight timing. This helps in monitoring flight occupancy.
- **Data Storage and Retrieval:** All booking records are automatically saved using file handling. Whenever the program starts again, previously saved booking information is loaded into the system.
- **Input Validation:** The system validates user input and prevents invalid entries such as letters in numeric fields, ensuring accurate and reliable data entry.
- **Exiting the Program:** After performing all required operations, users can safely exit the program. All booking records and modifications are automatically saved before the system closes.

4. References:

Sources:

- Studied from LinkedIn and Git-hub project codes
- Chat GPT for little bit idea .

5. Flow-Chart:



6. Project Working:

- Flight Ticket Booking
- Flight Ticket Cancellation
- View Booked Tickets
- Modify Existing Booking
- Booking Statistics Report
- Persistent Data Storage

7. Implementation Details:

Composition: Ticket class contains FlightSchedule class.

Single Inheritance: Passenger inherits from Person.

Multilevel Inheritance: VIPPassenger inherits from Passenger.

Multiple Inheritance: Ticket inherits from VIPPassenger and FlightDetails.

Runtime Polymorphism: Virtual function printPassengerType().

Compile-Time Polymorphism: Overloaded printReceipt() methods.

Friend Function: showSystemStatistics().

File Handling: AirportData.txt for saving records.

Input Validation: Prevents invalid numeric entries.

8. Learning Outcomes:

- Understanding of Object Oriented Programming.
- Practical implementation of inheritance and polymorphism.
- Experience with file handling and data persistence.
- Improved logical thinking and problem-solving skills.
- Understanding of real-world management systems.

9. Conclusion:

The Airport Management System project was successfully completed and achieved its objective of providing an efficient and user-friendly platform for managing airport operations. The system helps manage passenger records, flight details, ticket booking, cancellation, and updates in an organized manner, reducing manual work and minimizing errors. Through this project, we improved our understanding of C++ programming concepts such as file handling, functions, structures, and data management. It also enhanced our problem-solving, logical thinking, and teamwork skills. Overall, the project demonstrates how software solutions can be effectively used to simplify real-world airport management tasks and successfully meets its intended goals.

Appendix-Complete C++ Source Code

```
oop project.cpp
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 // Forward Declaration (Friend Function ke Liye)
8 class AirportManagementSystem;
9
10 // INPUT VALIDATION HELPER FUNCTION (BINA LIMITS LIBRARY KE)
11 int getValidatedInteger(string prompt) {
12     int value;
13     while (true) {
14         cout << prompt;
15         cin >> value;
16
17         // Agar user ne integer ki jagah koi letter ya character daal diya
18         if (cin.fail()) {
19             cin.clear(); // Error flag ko clear karta hai
20             cin.ignore(1000, '\n'); // Pehle se majood ghalat 1000 characters tak buffer saaf kar dega
21             cout << "[Error]: Invalid Input! Please enter a valid number (Letters are not allowed).\n" << endl;
22         } else {
23             cin.ignore(1000, '\n'); // Sahi input ke baad trailing newline saaf karne ke Liye
24             return value;
25         }
26     }
27 }
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56 public:
57     Person() {
58         name = "";
59         cnic = "";
60     }
61     virtual ~Person() {}
62
63     // Runtime Polymorphism implementation
64     void printPassengerType() {
65         cout << "[Passenger Type]: Standard Citizen Record" << endl;
66     }
67 };
68
69 // 3. SINGLE INHERITANCE (Person -> Passenger)
70 class Passenger : public Person {
71 protected:
72     string passport_number;
73
74 public:
75     Passenger() {
76         passport_number = "PK-A900";
77     }
78     ~Passenger() {}
79
80     void setPassengerDetails(string n, string c) {
81         name = n;
82     }
83
84 // 1. COMPOSITION CLASS
85 class FlightSchedule {
86 private:
87     string date;
88     int flight_time;
89
90 public:
91     FlightSchedule() {
92         date = "Not Set";
93         flight_time = 0;
94     }
95     ~FlightSchedule() {}
96
97     void setSchedule(string d, int t) {
98         date = d;
99         flight_time = t;
100    }
101
102    string getDate() { return date; }
103    int getFlightTime() { return flight_time; }
104 };
105
106 // 2. BASE CLASS (For Single & Multilevel Inheritance)
107 class Person {
108 protected:
109     string name;
110     string cnic;
111 }
```

```

82     cnic = c;
83 }
84 };
85
86 // 4. MULTILEVEL INHERITANCE (Person -> Passenger -> VIPPassenger)
87 class VIPPassenger : public Passenger {
88 protected:
89     int lounge_access_code;
90
91 public:
92     VIPPassenger() {
93         lounge_access_code = 101;
94     }
95     ~VIPPassenger() {}
96
97     void printPassengerType() {
98         cout << "[Passenger Type]: VIP Commercial Passenger (Lounge Access Granted)" << endl;
99     }
100 };
101
102 // 5. BASE CLASS 2 (For Multiple Inheritance)
103 class FlightDetails {
104 protected:
105     string departure_city;
106     string arrival_city;
107     string flight_class;
108     int fare;
109
110 public:
111     FlightDetails() {
112         departure_city = "";
113         arrival_city = "";
114         flight_class = "";
115         fare = 0;
116     }
117     ~FlightDetails() {}
118 };
119
120 // 6. MULTIPLE INHERITANCE & COMPOSITION (CORE TICKET CLASS)
121 class Ticket : public VIPPassenger, public FlightDetails {
122 private:
123     int ticket_id;
124
125     // Composition: Ticket "Has-A" FlightSchedule inside
126     FlightSchedule schedule;
127
128 public:
129     Ticket() {
130         ticket_id = 0;
131     }
132     ~Ticket() {}
133
134     void setTicketData(int id, string n, string c, string dep, string arr, string d, int t, string cls, int f) {
135         ticket_id = id;
136         setPassengerDetails(n, c);
137         departure_city = dep;
138         arrival_city = arr;
139         flight_class = cls;
140         fare = f;
141         schedule.setSchedule(d, t);
142     }
143
144     int getTicketId() { return ticket_id; }
145
146     // Friend Function ko access dene ke liye getter bana diya taake time check ho sake
147     int getFlightTimeFromSchedule() { return schedule.getFlightTime(); }
148
149     void updateBooking(string new_date, int new_time, string new_city) {
150         arrival_city = new_city;
151         schedule.setSchedule(new_date, new_time);
152     }
153
154     // Compile-time Polymorphism (Function Overloading)
155     void printReceipt() {
156         cout << "\n--- Flight Ticket Receipt ---" << endl;
157         cout << "Ticket ID:\t\t" << ticket_id << endl;
158         cout << "Passenger Name:\t" << name << endl;
159         cout << "CNIC:\t\t\t" << cnic << endl;
160         cout << "From:\t\t\t" << departure_city << endl;
161         cout << "To:\t\t\t" << arrival_city << endl;

```

```

163     cout << "Flight Time:\t\t" << schedule.getFlightTime() << " Hours" << endl;
164     cout << "Class:\t\t\t" << flight_class << endl;
165     cout << "Fare:\t\t\tRs. " << fare << endl;
166 }
167
168 void printReceipt(string layout_type) {
169     cout << "[" << layout_type << "]" ID: " << ticket_id << " | Passenger: " << name
170     << " | Route: " << departure_city << " to " << arrival_city << " | Time: " << schedule.getFlightTime() << endl;
171 }
172
173 // File Handling Methods
174 void writeToFile(ofstream &file) {
175     file << ticket_id << "\n" << name << "\n" << cnic << "\n"
176     << departure_city << "\n" << arrival_city << "\n" << schedule.getDate() << "\n"
177     << schedule.getFlightTime() << "\n" << flight_class << "\n" << fare << "\n";
178 }
179
180 void readFromFile(istream &file) {
181     string d_input;
182     int t_input;
183     file >> ticket_id;
184     file.ignore();
185     getline(file, name);
186     getline(file, cnic);
187     getline(file, departure_city);
188     getline(file, arrival_city);
189     getline(file, d_input);
190     file >> t_input;
191     file.ignore();
192     getline(file, flight_class);
193     file >> fare;
194     file.ignore();
195     schedule.setSchedule(d_input, t_input);
196 }
197 };
198
199 // 7. AIRPORT SYSTEM CONTROLLER CLASS
200 class AirportManagementSystem {
201 private:
202     Ticket passenger_tickets[1000]; // Array Storage
203     int total_bookings;
204
205 public:
206     AirportManagementSystem() {
207         total_bookings = 0;
208     }
209     ~AirportManagementSystem() {}
210
211     // Friend Function Declaration
212     friend void showSystemStatistics( AirportManagementSystem &sys);
213
214     void loadData() {
215         ifstream file("AirportData.txt");
216         if (!file) return;
217         file >> total_bookings;
218         file.ignore();
219         for (int i = 0; i < total_bookings; i++) {
220             passenger_tickets[i].readFromFile(file);
221         }
222         file.close();
223     }
224
225     void saveData() {
226         ofstream file("AirportData.txt");
227         if (!file) return;
228         file << total_bookings << endl;
229         for (int i = 0; i < total_bookings; i++) {
230             passenger_tickets[i].writeToFile(file);
231         }
232         file.close();
233     }
234
235     void bookTicket() {
236         if (total_bookings >= 1000) {
237             cout << "[Notice]: Database storage capacity maxed out!" << endl;
238             return;

```



```

267 cout << "Enter Date (dd/mm/yyyy): ";
268 cin >> date_input;
269
270 do {
271     cout << "Select Flight Time:\n1. 1200\n2. 1600" << endl;
272     select2 = getValidatedInteger("Choose option: ");
273     if (select2 == 1) time_input = 1200;
274     else if (select2 == 2) time_input = 1600;
275     else cout << "[Warning]: Option doesn't exist.\n" << endl;
276 } while (select2 < 1 || select2 > 2);
277
278 do {
279     cout << "Select Class:\n1. First Class (50000)\n2. Business Class (35000)\n3. Economy Class (25000)" << endl;
280     select3 = getValidatedInteger("Choose option: ");
281     if (select3 == 1) { class_input = "First"; fare_input = 50000; }
282     else if (select3 == 2) { class_input = "Business"; fare_input = 35000; }
283     else if (select3 == 3) { class_input = "Economy"; fare_input = 25000; }
284     else cout << "[Warning]: Selected dynamic configuration profile invalid.\n" << endl;
285 } while (select3 < 1 || select3 > 3);
286
287 cout << "Enter Passenger Name: ";
288 getline(cin, p_name);
289
290 cout << "Enter CNIC: ";
291 cin >> cnic_input;
292
293 int select, select1, select2, select3, select4;
294 string dep_city, arr_city, date_input, p_name, cnic_input, class_input;
295 int time_input = 0, fare_input = 0;
296
297 do {
298     cout << "Select Departure City:\n1. Islamabad\n2. Peshawar\n3. Lahore" << endl;
299     select = getValidatedInteger("Choose valid option (1-3): ");
300     if (select == 1) dep_city = "Islamabad";
301     else if (select == 2) dep_city = "Peshawar";
302     else if (select == 3) dep_city = "Lahore";
303     else cout << "[Warning]: Choice out of bounds! Try again.\n" << endl;
304 } while (select < 1 || select > 3);
305
306 do {
307     cout << "Select Arrival City:\n1. Islamabad\n2. Peshawar\n3. Lahore" << endl;
308     select1 = getValidatedInteger("Choose valid option (1-3): ");
309     if (select1 == 1) arr_city = "Islamabad";
310     else if (select1 == 2) arr_city = "Peshawar";
311     else if (select1 == 3) arr_city = "Lahore";
312     else cout << "[Warning]: Choice out of bounds! Try again.\n" << endl;
313
314     if (dep_city == arr_city && select1 >= 1 && select1 <= 3) {
315         cout << "[Error]: Departure and Arrival cities cannot be the same! Choose another destination.\n" << endl;
316     }
317 } while (select1 < 1 || select1 > 3 || dep_city == arr_city);
318
319 int generated_id = 1000 + total_bookings + select2;
320
321 cout << "\nPress 1 to Confirm Ticket Booking: ";
322 select4 = getValidatedInteger("Confirm code: ");
323
324 if (select4 == 1) {
325     passenger_tickets[total_bookings].setTicketData(generated_id, p_name, cnic_input, dep_city, arr_city, date_input, time_input,
326     passenger_tickets[total_bookings].printReceipt();
327     total_bookings++;
328     cout << "\n>>> Flight Ticket Successfully Manifested!" << endl;
329 } else {
330     cout << "\n>>> Booking Cancelled." << endl;
331 }
332 }
333
334 void cancelTicket() {
335     if (total_bookings == 0) {
336         cout << "\n[Notice]: No bookings available to cancel." << endl;
337         return;
338     }
339     int id_to_cancel = getValidatedInteger("Enter Ticket ID to Cancel: ");
340     for (int i = 0; i < total_bookings; i++) {
341         if (passenger_tickets[i].getTicketId() == id_to_cancel) {
342             for (int j = i; j < total_bookings - 1; j++) {

```

```

319         total_bookings--;
320         cout << "Flight Ticket ID " << id_to_cancel << " Cancelled Successfully." << endl;
321         return;
322     }
323 }
324 cout << "Ticket ID not found in database records." << endl;
325 }
326
327 void viewAllTickets() {
328     if (total_bookings == 0) {
329         cout << "\nNo active passenger manifests found." << endl;
330         return;
331     }
332     cout << "\n===== CURRENT REGISTERED MANIFESTS =====" << endl;
333     for (int i = 0; i < total_bookings; i++) {
334         passenger_tickets[i].printReceipt("SHORT MANIFEST");
335     }
336     cout << "===== " << endl;
337 }
338
339 void modifyBooking() {
340     if (total_bookings == 0) {
341         cout << "\n[Notice]: Database empty. Nothing to modify." << endl;
342         return;
343     }
344     int id_to_search = getValidatedInteger("Enter Ticket ID to Modify: ");
345     for (int i = 0; i < total_bookings; i++) {
346         if (passenger_tickets[i].getTicketId() == id_to_search) {
347             string new_date, new_city;
348             int select2, select1, new_time;
349
350             cout << "Enter New Date (dd/mm/yyyy): ";
351             cin >> new_date;
352
353             do {
354                 cout << "Select Time:\n1. 1200\n2. 1600" << endl;
355                 select2 = getValidatedInteger("Choose options (1-2): ");
356                 new_time = (select2 == 1) ? 1200 : 1600;
357             } while(select2 < 1 || select2 > 2);
358
359             do {
360                 cout << "Select New Arrival City:\n1. Islamabad\n2. Peshawar\n3. Lahore" << endl;
361                 select1 = getValidatedInteger("Choose option (1-3): ");
362                 new_city = (select1 == 1) ? "Islamabad" : (select1 == 2 ? "Peshawar" : "Lahore");
363             } while(select1 < 1 || select1 > 3);
364
365             passenger_tickets[i].updateBooking(new_date, new_time, new_city);
366             cout << "\nFlight Booking Updated Successfully!" << endl;
367             return;
368         }
369     }
370     cout << "Invalid Ticket ID input entry." << endl;
371 }
372 };
373
374 // 8. FRIEND FUNCTION IMPLEMENTATION (TIMING TRACKING ADDED!)
375 void showSystemStatistics( AirportManagementSystem &sys) {
376     int count_1200 = 0;
377     int count_1600 = 0;
378
379     // Loop chala kar check kar rahe hain ke kaunsi ticket kis time par booked hai
380     for (int i = 0; i < sys.total_bookings; i++) {
381         if (sys.passenger_tickets[i].getFlightTimeFromSchedule() == 1200) {
382             count_1200++;
383         } else if (sys.passenger_tickets[i].getFlightTimeFromSchedule() == 1600) {
384             count_1600++;
385         }
386     }
387
388     cout << "\n===== BOOKING REPORT =====" << endl;
389     cout << " Total Number of Booked Tickets : " << sys.total_bookings << " / 1000" << endl;
390     cout << "-----" << endl;
391     cout << " Active Bookings for 12:00 PM Flight : " << count_1200 << " Tickets" << endl;
392     cout << " Active Bookings for 04:00 PM Flight : " << count_1600 << " Tickets" << endl;
393     cout << "===== " << endl;

```

```

396 // MAIN EXECUTIVE INTERFACE TERMINAL
397 int main()
398     AirportManagementSystem system;
399     system.loadData();
400
401     int option;
402     char choice;
403
404     cout << "===== Welcome to Airport Management System =====" << endl;
405     do {
406         cout << "\n===== AIRPORT INTERACTION MENU =====" << endl;
407         cout << "1. Book Flight Ticket" << endl;
408         cout << "2. Flight Ticket Cancellation" << endl;
409         cout << "3. View Booked Tickets" << endl;
410         cout << "4. Modify Flight Booking" << endl;
411         cout << "5. Number of Booked Tickets" << endl;
412         cout << "6. Exit Terminal System" << endl;
413
414         option = getValidatedInteger("Choose menu option: ");
415
416         switch (option) {
417             case 1: system.bookTicket(); break;
418             case 2: system.cancelTicket(); break;
419             case 3: system.viewAllTickets(); break;
420             case 4: system.modifyBooking(); break;
421             case 5: showSystemStatistics(system); break; // Live tracking report call
422             case 6: break;
423             default: cout << "[Warning]: Invalid entry! Please select options between 1 and 6." << endl;
424         }
425         if (option == 6) break;
426
427         cout << "\nPress Y/y for Main Menu or any other key to close: ";
428         cin >> choice;
429     } while (choice == 'Y' || choice == 'y');
430
431     system.saveData();
432     cout << "\nTerminal Session Secured. Airport Database Saved." << endl;
433     return 0;
434

```

Output Window:

Ticket Booking:

```
C:\Users\DELL Latitude 7430\I  X  +  v

===== Welcome to Airport Management System =====

===== AIRPORT INTERACTION MENU =====
1. Book Flight Ticket
2. Flight Ticket Cancellation
3. View Booked Tickets
4. Modify Flight Booking
5. Number of Booked Tickets
6. Exit Terminal System
Choose menu option: 1
Select Departure City:
1. Islamabad
2. Peshawar
3. Lahore
Choose valid option (1-3): 1
Select Arrival City:
1. Islamabad
2. Peshawar
3. Lahore
Choose valid option (1-3): 2
Enter Date (dd/mm/yyyy): 10/3/2036
Select Flight Time:
1. 1200
2. 1600
Choose option: 1
Select Class:
1. First Class (50000)
2. Business Class (35000)
3. Economy Class (25000)
Choose option: 1
Enter Passenger Name: Shahzaib
Enter CNIC: 6110197914483

Press 1 to Confirm Ticket Booking: Confirm code: 1

--- Flight Ticket Receipt ---
Ticket ID:          1001
Passenger Name: Shahzaib
CNIC:               6110197914483
From:               Islamabad
To:                 Peshawar
Date:               10/3/2036
Flight Time:        1200 Hours
Class:              First
Fare:               Rs. 50000

>>> Flight Ticket Successfully Manifested!

Press Y/y for Main Menu or any other key to close: |
```


View Booked Tickets:

```
===== AIRPORT INTERACTION MENU =====
1. Book Flight Ticket
2. Flight Ticket Cancellation
3. View Booked Tickets
4. Modify Flight Booking
5. Number of Booked Tickets
6. Exit Terminal System
Choose menu option: 3

===== CURRENT REGISTERED MANIFESTS =====
[SHORT MANIFEST] ID: 1001 | Passenger: Shahzaib | Route: Islamabad to Peshawar | Time: 1200
=====
```

Number Of Booked Tickets:

```
===== BOOKING REPORT =====
Total Number of Booked Tickets : 1 / 1000
-----
Active Bookings for 12:00 PM Flight : 1 Tickets
Active Bookings for 04:00 PM Flight : 0 Tickets
=====
```

Update Tickets:

```
===== AIRPORT INTERACTION MENU =====
1. Book Flight Ticket
2. Flight Ticket Cancellation
3. View Booked Tickets
4. Modify Flight Booking
5. Number of Booked Tickets
6. Exit Terminal System
Choose menu option: 4
Enter Ticket ID to Modify: 1001
Enter New Date (dd/mm/yyyy): 20/29/2046
Select Time:
1. 1200
2. 1600
Choose options (1-2): 2
Select New Arrival City:
1. Islamabad
2. Peshawar
3. Lahore
Choose option (1-3): 3

Flight Booking Updated Successfully!
```

Flight Ticket Cancellation:

```
===== AIRPORT INTERACTION MENU =====  
1. Book Flight Ticket  
2. Flight Ticket Cancellation  
3. View Booked Tickets  
4. Modify Flight Booking  
5. Number of Booked Tickets  
6. Exit Terminal System  
Choose menu option: 2  
Enter Ticket ID to Cancel: 1001  
Flight Ticket ID 1001 Cancelled Successfully.
```

Exiting Menu:

```
===== AIRPORT INTERACTION MENU =====  
1. Book Flight Ticket  
2. Flight Ticket Cancellation  
3. View Booked Tickets  
4. Modify Flight Booking  
5. Number of Booked Tickets  
6. Exit Terminal System  
Choose menu option: 6  
  
Terminal Session Secured. Airport Database Saved.
```